

Question 1:

1] Abstract class is a class that cannot be initiated (create an object from it), it might contain different types of attributes and methods that subclasses can inherit from/implement and make use of it uniquely and individually.

```
from abc import ABC, abstractmethod

class Animal(ABC):

    @abstractmethod
    def make_sound(self):
        pass
```

Abstract method : When adding @abstractmethod - from a certain library as above-before a method, this is an abstract method, subclass must implement the method from parent class.

-----Following the code above-----

```
class Dog(Animal):

    def make_sound(self):
        print("Woof")
```

3] *Partial vs full Abstraction* : Partial abstraction provide some methods that are already implemented and can be used directly by child classes, while other methods are abstract and need to be implemented by the sub classes. While in full abstraction all methods are abstract.

3] Python does not have a dedicated interface keyword like java does ,however, same behavior can be achieved through abstract methods.

4]When trying to initiate a class that has an unimplemented abstract method -like the code above- when we **Run** it, it will show TypeError.

1.2

1]Duck typing is a concept in programming where an object's suitability is determined by the presence of specific methods and properties we are looking for , rather than its explicit class or inheritance. and it comes from “if it walks like a duck and quacks like a duck , it is a duck”

2] We are able to type similar methods/functions without a common parent class.

3] In the overriding method , inheritance for a sub class is a must in addition to its own unique implementation.

4] each one can be called individually .

```
class Dog:  
    def speak(self):  
        print("Woof")
```

```
class Cat:  
    def speak(self):  
        print("Meow")
```